

DART

basics

IT 331 – APPLICATION DEVELOPMENT AND EMERGING TECHNOLOGIES

DART

basics

dart programming language

- 🎯 an open-source programming language developed by Google
- 🎯 used for building web, server, and mobile applications
- 🎯 compilers: Dart VM and dart2js (for JavaScript conversion)

DART

basics

features



easy to understand

FAMILIAR SYNTAX FOR C# AND JAVA USERS.



object-oriented

FOLLOWS PRINCIPLES SIMILAR TO C++ AND JAVA.

DART

basics

features



open-source

POPULAR AMONG INDIVIDUALS & ORGANIZATIONS.



browser support

RUNS ON ALL MODERN WEB BROWSERS VIA DART2JS.

DART

basics

features



type safe

COMBINES STATIC AND RUNTIME CHECKS



asynchronous programming

EFFICIENT HANDLING OF ASYNCHRONOUS TASKS

your first dart program

ENTER THE FOLLOWING CODE:

```
main() {  
    print("Welcome to IT 331: Application  
    Development and Emerging Technologies.");  
}
```

VARIABLES

static variables

output

```
100
0.2
false
0
IT 331: AppDev
```

```
void main()
{
    // Declaring and initialize a variable
    int var1 = 100;

    // Declaring another variable
    double var2 = 0.2;
    // must declare double a value or it will throw error
    bool var3 = false;
    // must declare Boolean a value or it will throw error

    // Declaring multiple variables
    String var4 = "0", var5 = "IT 331: AppDev";

    // Printing values of all the variables
    print(var1); // Print 100
    print(var2); // Print 0.2
    print(var3); // Print default string value
    print(var4); // Print default bool value
    print(var5); // Print IT 331: AppDev
}
```


dynamic variables

IMPLICITLY STORES ANY VALUE AT RUNTIME.

output

pretty
3.14157

```
void main()  
{  
    // Assigning value to variable var1  
    dynamic var1 = "pretty";  
  
    // Printing variable var1  
    print(var1);  
  
    // Reassigning the data to variable  
    // and printing it  
    var1 = 3.14157;  
    print(var1);  
}
```

final keyword

CAN ONLY BE SET ONCE, AND
INITIALIZED WHEN ACCESSED.

output

```
AppDev is fun!  
Dart is fun!
```

```
void main() {  
    // Assigning value to var1 variable without datatype  
    final var1 = "AppDev is fun!";  
    // Printing variable var1  
    print(var1);  
  
    // Assigning value to var2 variable with datatype  
    final String var2 = "Dart is fun!";  
    // Printing variable var2  
    print(var2);  
}
```


const keyword

VALUE MUST BE KNOWN BEFORE
RUNTIME.

output

```
AppDev is fun!  
Dart is fun too!
```

```
void main() {  
    // Assigning value to var1 variable w/o datatype  
    const var1 = "AppDev is Fun!";  
    // Printing variable var1  
    print(var1);  
  
    // Assigning value to var2 variable with datatype  
    const var2 = "Dart is fun too!";  
    // Printing variable var2  
    print(var2);  
}
```

null safety

A variable cannot be assigned as null until it is defined that the variable can store the null value in it.

```
void main() {  
    int x = 10;  
    x = NULL;  
    print(x);  
}
```

output

```
variables.dart:18:8: Error: Undefined name 'NULL'.  
    x = NULL;
```


assign null values

The '?' character is used in declaring a nullable variable to its type. With that, the variable declared will store null as its value by default.

```
void main() {  
    int? x;  
    x = null;  
    print(x);  
}
```

output

null

INPUT/OUTPUT

input/output

The `.readLineSync()` function is used for taking standard inputs from the user through the console.

Also, the library `dart:io` must be imported.

```
// importing dart:io file
import 'dart:io';

void main()
{
    print("Enter your name: ");
    // Reading name
    String? name = stdin.readLineSync();
    // null safety in name string

    // Printing the name
    print("Hello, $name! \nWelcome to IT 331: AppDev!!");
}
```

output

```
Enter your name:
Juan Dela Cruz
Hello, Juan Dela Cruz!
Welcome to IT 331: AppDev!!
```


getting integer input

output

```
Enter a number:  
92  
The number inputted is: 92
```

```
// Importing dart:io file  
import 'dart:io';  
  
void main()  
{  
    // Asking a number  
    print("Enter a number:");  
  
    // Scanning number  
    int? n = int.parse(stdin.readLineSync()!);  
    // Here ? and ! are for null safety  
  
    // Printing that number  
    print("The number inputted is: $n");  
}
```


standard output

`print( )` statement brings the cursor to the next line.

`stdout.write( )` remains at the same line

```
import 'dart:io';

void main()
{
    // Printing in first way - using print
    print("Hello world! This is printed using print statement");

    // Printing in second way - using stdout.write()
    stdout.write("Meanwhile, this is printed using stdout.write()");
    stdout.write("and here's another using stdout.write()");
}
```

output

```
Hello world! This is printed using print statement
Meanwhile, this is printed using stdout.write() and here's another using stdout.write()
```

DATA TYPES

number

DATA TYPE USED FOR
HOLDING NUMERIC VALUES

output

```
2
1.5
Product = 50.24
```

```
// Dart Program to demonstrate the Number Data Type
void main() {

    // declare an integer
    int num1 = 2;
    // declare a double value
    double num2 = 1.5;
    // print the values
    print(num1);
    print(num2);

    var a = num.parse("16");
    var b = num.parse("3.14");

    var product = a * b;
    print("Product = ${product}");
}
```


string

REPRESENTING A SEQUENCE
OF CHARACTERS, UTF-16
CODE UNITS TO BE SPECIFIC

output

```
This is a string  
Dart is Fun! <3
```

```
// Dart Program to demonstrate String Data Type
```

```
void main() {  
    // Declaration of String type  
    String str1 = "This is a string";  
    String str2 = 'Dart is '  
    String str3 = 'Fun! <3';  
  
    print (str1);  
    print (str2 + str3);  
}
```

boolean

REPRESENTS TWO VALUES:
TRUE AND FALSE

output

```
abc == ABC:  
false
```

```
// Dart Program to demonstrate  
// Boolean Data Type  
void main() {  
    String str1 = 'abc';  
    String str2 = 'ABC';  
  
    bool val = (str1==str2);  
    print("$str1 == $str2: " );  
    print (val);  
}
```


lists

USED FOR REPRESENTING
A COLLECTION OF
OBJECTS, IN AN ORDERED
MANNER

```
void main()  
{  
    List<String> list1 = new List<String>.filled(3,"default");  
    list1[0] = 'Juan';  
    list1[1] = 'Maria';  
    list1[2] = 'Cruz';  
  
    print(list1);  
    print(list1[0]);  
}
```

output

```
[Juan, Maria, Cruz]  
Juan
```

maps

A DYNAMIC COLLECTION THAT IS A PAIRING OF A KEY AND VALUE, AND THAT COULD BE OF ANY TYPE

```
void main() {  
    Map map1 = new Map();  
    map1[1] = 'Jell';  
    map1[2] = 'Charlie';  
    map1[3] = 'Jhomari';  
    map1[4] = 'John';  
    print(map1);  
}
```

output

```
{1: Jell, 2: Charlie, 3: Jhomari, 4: John}
```

CONTROL FLOW

switch-case

SIMPLIFIED
STATEMENT

NESTED

IF-ELSE

output

Number is 1.

```
void main()
{
    int val = 1;
    switch (val) {
        case 1: {
            print("Number is 1.");
        } break;
        case 2: {
            print("Number is 2.");
        } break;
        case 3: {
            print("Number is 3.");
        } break;
        default: {
            print("This is default case.");
        } break;
    }
}
```


for loop

- (1) INITIALIZATION,
- (2) CONDITION
- (3) BODY OF LOOP,
- (4) TEST EXPRESSION

output

```
AppDev is fun!  
AppDev is fun!  
AppDev is fun!  
AppDev is fun!  
AppDev is fun!
```

```
// Printing AppDev is fun! five times  
void main()  
{  
    for (int i = 0; i < 5; i++) {  
        print('AppDev is fun!');  
    }  
}
```

for...in loop

TAKES AN EXPRESSION OR OBJECT AS ITS ITERATOR

```
void main()  
{  
    var sample = [ 'Maurice', 'Jayson', 'Arjonel', 'Jell', 'Edbert' ];  
    for (String str in sample) {  
        print(str);  
    }  
}
```

output

```
Maurice Oliver  
Jayson  
Arjonel  
Jell  
Edbert
```

for...each loop

ITERATES OVER ALL THE ELEMENTS
IN A CONTAINER OR A COLLECTION,
AND THE ELEMENTS ARE PASSED TO
SOME SPECIFIC METHOD

output

```
values in sample:  
28  
21  
27  
16  
29
```

```
void main() {  
    var sample = [28, 21, 27, 16, 29];  
    print("values in sample: ");  
    sample.forEach((var num)=> print(num));  
}
```


while loop

PRE-CHECKING LOOP

```
void main()  
{  
    var sample = 4;  
    int i = 1;  
    while (i <= sample) {  
        print('App Dev and Dart are fun!');  
        i++;  
    }  
}
```

output

```
App Dev and Dart are fun!  
App Dev and Dart are fun!  
App Dev and Dart are fun!  
App Dev and Dart are fun!
```


do-while

EXECUTES THE STATEMENT, AND THEN CHECKS THE CONDITION.

```
void main()
{
    var sample = 4;
    int i = 1;
    do {
        print('Pikachu');
        i++;
    } while (i <= sample);
}
```

output

```
Pikachu
Pikachu
Pikachu
Pikachu
```

```
void main()
{
    var sample = 4;
    int i = 7;
    do {
        print('Pikachu');
        i++;
    } while (i <= sample);
}
```

output

```
Pikachu
```

FUNCTIONS

anonymous functions

SELF-CONTAINED BLOCKS OF CODE THAT CAN BE PASSED AROUND IN THE CODE AS A FUNCTION PARAMETER

```
// Dart program to illustrate Anonymous functions
void main()
{
  var list = ["Maurice Oliver", "Jayson", "Arjonel", "Jell", "Edbert"];
  print("Anonymous function in Dart");
  list.forEach((item) {
    print('${list.indexOf(item)} : $item');
  });
}
```

output

```
Anonymous function in Dart
0 : Maurice Oliver
1 : Jayson
2 : Arjonel
3 : Jell
4 : Edbert
```

main function

MOST IMPORTANT PREDEFINED METHOD IN DART

```
main(){  
    print("This is the main method, the entry point of Dart. :D");  
}
```

output

```
This is the main method, the entry point of Dart. :D
```


arguments

HOW ARGUMENTS ARE PASSED
INSIDE THE MAIN FUNCTION

```
main(List<String> arguments){  
    //printing the arguments along with length  
    print(arguments.length);  
    print(arguments);  
}
```

command

```
dart arguments.dart Jigglypuff Psyduck Squirtle
```

output

```
3  
[Jigglypuff, Psyduck, Squirtle]
```

collection methods

COLLECTIONS SUCH AS LIST, SET, AND MAP
SHARE A COMMON FUNCTIONALITY

```
void main(){  
    var oppa_line = ['Maurice Oliver', 'Jerome'];  
    var maknae_line = ['Fatima Marie', 'Gerald', 'Arjone1'];  
    print(oppa_line.isEmpty);  
    print(maknae_line.isNotEmpty);  
}  
print(arguments);  
}
```

output

```
false  
true
```

forEach()

USED TO APPLY A FUNCTION IN EACH ITEM
FROM THE LIST, SET, OR MAP

```
void main(){  
    var family = ['Loid', 'Yor', 'Anya', 'Bond'];  
    var member = family.map((member) => member.toUpperCase());  
    member.forEach(print);  
}
```

output

```
LOID  
YOR  
ANYA  
BOND
```


where

```
void main(){  
  var trainer = ['Ash', 'Misty', 'Brock', 'May', 'Drew'];  
  // Pikachu is not a trainer  
  bool isTrainer(String trName) =>  
    trName == 'Pikachu';  
  // Use where() to find only the items that return true from the provided function.  
  // Use any() to check whether at least one item in the collection satisfies a  
  condition.  
  print(trainer.any(isTrainer));  
  // Use every() to check whether all the items in a collection satisfy a condition.  
  print(!trainer.every(isTrainer));  
}
```

output

```
false  
true
```


getters/setters

ALSO KNOWN AS ACCESSORS AND MUTATORS, AND ARE USED FOR INITIALIZATION AND RETRIEVING THE CLASS FIELD VALUES.

```
// Dart Program in Dart to illustrate getters and setters
```

```
class Student {  
  String name = '';  
  int age = 0;
```

```
  String get stud_name {  
    return name;  
  }
```

```
  void set stud_name(String name) {  
    this.name = name;  
  }
```

```
void set stud_age(int age) {  
    if(age <= 0) {  
        print("Age should be greater than 5");  
    } else {  
        this.age = age;  
    }  
}
```

```
int get stud_age {  
    return age;  
}  
}
```

```
void main() {  
    Student s1 = new Student();  
    s1.stud_name = 'Anya';  
    s1.stud_age = 6;  
    print(s1.stud_name);  
    print(s1.stud_age);  
}
```

output

Anya
6

classes and objects

THE BLUEPRINT AND THE EMBODIMENT

```
class Trainer {  
    String name = "Jerome";  
  
    // Getter for the name field  
    String get tname {  
        return name;  
    }  
  
    // Setter for the name field  
    void set tname(String tname) {  
        this.name = tname;  
    }  
}
```

```
// Method to greet the trainer by printing their name
void greetTrainer() {
    print("Welcome, Trainer $name");
    // This should print the actual name
}

void main() {
    Trainer tr1 = new Trainer();
    tr1.greetTrainer(); // This will print "Jerome"

    Trainer tr2 = new Trainer();
    tr2.tname = "Gerald";
    tr2.greetTrainer(); // This will print "Gerald"
} output
```

```
Welcome, Trainer Jerome
Welcome, Trainer Gerald
```


DART OOPS

static keyword

Used to manage memory of global data members, and can be applied to the fields and methods of the class.

Static variables and methods are part of the class instead of a specific instance or object.

```
import 'dart:io';  
// Static methods in Dart  
class Pokemon {  
  static var pk_type;  
  var pk_name;  
  int pk_level = 5;  
  
  // Function to show details of the Pokemon  
  showDetails() {  
    print("Name: ${pk_name}");  
    print("Level: ${pk_level}");  
    print("Type: ${pk_type}");  
  }  
}
```

output

Name: Milotic
Level: 54
Type: Water

Name: Swampert
Level: 67
Type: Water

```
//Main function
void main() {
    Pokemon pk1 = new Pokemon();
    Pokemon pk2 = new Pokemon();
    Pokemon.pk_type = "Water";

    pk1.pk_name = 'Milotic';
    pk1.pk_level = 54;
    pk1.showDetails();

    for(int ctr = 1; ctr <= 30; ctr++){
        stdout.write("-");
    }

    stdout.write("\n");

    pk2.pk_name = 'Swampert';
    pk2.pk_level = 67;
    pk2.showDetails();
}
```


super keyword

Signifies that it is the object of an immediate ancestor class, allowing access to superclass properties and methods.

output

```
Inside the parent constructor.  
Inside the child constructor.  
Instance of 'SubClass'
```

```
class SuperClass {  
    // Creating parent constructor  
    SuperClass()  
    {  
        print("Inside the parent constructor.");  
    }  
}  
  
class SubClass extends SuperClass {  
    // Creating child constructor  
    SubClass()  
    {  
        print("Inside the child constructor.");  
    }  
}  
  
void main()  
{  
    SubClass sc = new SubClass();  
    print(sc);  
}
```

instance methods

If a method is not declared as static, it is classified as an instance method in a class, meaning, they are allowed to access instance variables. An object must be created first, in order to call the method of this class.

```
// Creating Class named Number
class Number {
    // Declaring instance variable
    int num1 = 0;
    int num2 = 0;

    // Creating instance method name getSum with two parameters
    void getSum(int num3, int num4)
    {
        // Using this to set the values of the input to instance variable
        this.num1 = num3;
        this.num2 = num4;
    }
}
```



```
        // Printing the result
        print('The sum of numbers is ${num1 + num2}');
    }
}

void main()
{
    // Creating instance of the class Number(Creating Object)
    Number sample = new Number();

    // Calling the method getSum with the use of object
    sample.getSum(21, 27);
}
```

output

The sum of numbers is 48

class methods

Declared with the static keyword, meaning they cannot access non-static variables and won't be able to invoke non-static methods of the class.

```
// Creating Class named Number
class Number {

    // Creating a class method name sum with two parameters
    static void sum(int num1, int num2)
    {
        // Printing the result
        print('The sum of numbers is: ${num1 + num2}');
    }
}

void main()
{
    // Calling the method sum without
    // the use of object, with class name
    Number.sum(16, 29);
}
```

output

The sum of numbers is: 45

method overriding

HAPPENS WHEN A DERIVED CLASS IS TRYING TO OVERRIDE THE BASE CLASS'S METHOD.

```
// Dart Program to illustrate the method overriding concept

class BaseClass {
  // Creating a method
  void show(){
    print("This is class BaseClass.");
  }
}

class FirstDerivedClass extends BaseClass {
  // Overriding show method
  void show(){
    print("This is class FirstDerivedClass child of BaseClass.");
  }
}
```



```
class SecondDerivedClass extends BaseClass {
    // Overriding show method
    void show(){
        print("This is class SecondDerivedClass child of BaseClass.");
    }
}

void main() {
    // Creating objects of both the classes
    BaseClass base_var = new BaseClass();
    FirstDerivedClass derived1 = new FirstDerivedClass();
    SecondDerivedClass derived2 = new SecondDerivedClass();

    // Calling the same function from both the classes object to show method
    // overriding
    base_var.show();
    derived1.show();
    derived2.show();
}
```

output

```
This is class BaseClass.
This is class FirstDerviedClass child of BaseClass.
This is class SecondDerivedClass child of BaseClass.
```


abstract classes

CONTAINS ONE OR MORE ABSTRACT METHODS IN THEM (BUT CAN ALSO CONTAIN CONCRETE METHODS)

```
// Creating Abstract Class
abstract class Pokemon {
    // Creating Abstract Method
    void fight();
}

// Class WaterType Inheriting Pokemon class
class WaterType extends Pokemon {
    // Overriding method
    @override
    void fight()
    {
        print("WaterType uses Bubblebeam!");
    }
}
```

```
// Class FireType Inheriting Pokemon class
class FireType extends Pokemon {
    // Overriding method again
    @override
    void fight()
    {
        print("FireType uses Flamethrower!");
    }
}

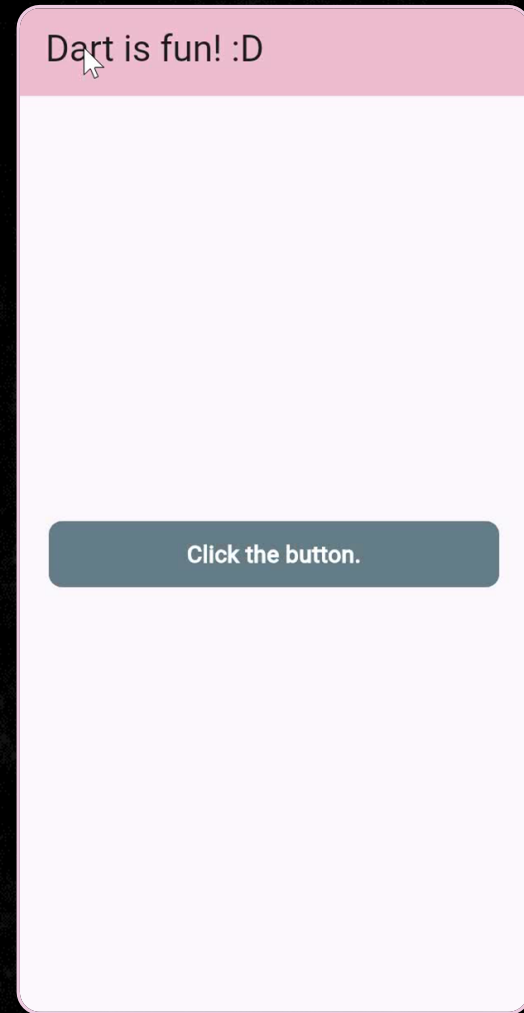
void main()
{
    WaterType squirtle = new WaterType();
    squirtle.fight();
    FireType charmander = new FireType();
    charmander.fight();
}
```

output

```
WaterType used Bubblebeam!
FireType used Flamethrower!
```


Builder class

USED WHEN CREATING A NEW
WIDGET IN FLUTTER.



callable class

ALLOWS THE INSTANCE OF THE CLASS TO BE CALLED AS A FUNCTION.

```
// Creating Class Trainer
class Trainer {
    // Defining call method which create the class callable
    String call(String tr1, String tr2, String tr3) => 'Hello trainers: $tr1, $tr2, $tr3!';
}

// Main Function
void main() {
    // Creating instance of class
    var trainer_input = Trainer();
    // Calling the class through its instance
    var trainer_output = trainer_input('Fatima Marie', 'Jerome', 'Gerald James');
    // Printing the output
    print(trainer_output);
}
```

output

Hello trainers: Fatima Marie, Jerome, Gerald James!

interfaces

OUTLINES THE STRUCTURE A CLASS SHOULD ADHERE TO, IF IT IS IMPLEMENTING THAT INTERFACE.

```
// Class DragonType (Interface 1)
class DragonType {
    void DragonPulse() {
        print("Dragon Pulse was used!");
    }
}

// Class FlyingType (Interface 2)
class FlyingType {
    void SkyAttack() {
        print("Sky Attack was used!");
    }
}
```



```
// Class Altaria implementing DragonType, FlyingType.
class Altaria implements DragonType, FlyingType {
    void DragonPulse() {
        print("Altaria used Dragon Pulse!");
    }

    void SkyAttack() {
        print("Altaria used Sky Attack!");
    }
}

void main(){
    // Creating Object of the class Altaria
    Altaria al= new Altaria();

    // Calling method (After Implementation )
    print("Altaria is ready for battle!");
    al.DragonPulse();
    al.SkyAttack();
}
```

output

```
Altaria is ready for battle!
Altaria used Dragon Pulse!
Altaria used Sky Attack!
```


mixins

PROVIDE A WAY TO REUSE
METHODS ACROSS DIFFERENT
CLASS HIERARCHIES WITHOUT
USING MULTIPLE INHERITANCES

```
// mixin with name Attack
mixin Attack {
  void HyperBeam(){
    print('The pokemon used Hyper Beam!');
  }
}

// mixin with name level
mixin level {
  void number(){
    print('Level: 70');
  }
}

// mixin type used with keyword
class Gyarados with Attack, level{
  @override
  void HyperBeam(){
    print('Gyarados used Hyper Beam!');
  }
}
```

```
void main(){  
    var gy1 = Gyarados();  
    gy1.HyperBeam();  
    gy1.number();  
}
```

output

Gyarados used Hyper Beam!
Level: 70

THANK
you